Labo 08 — Labo pratique : configurer sans reconstruire, cacher sans se mentir

Objectif : piloter la même image `api:1.0` par variables, mesurer la priorité des sources, voir de vos yeux où un secret apparaît (logs, `ps`, `inspect`, `env`), puis le faire disparaître avec `podman secret` et un montage de secret au build.

Prérequis — Labos 01 à 07 terminés. Image `api:1.0` présente (labo 07, étape 0) ; `postgres:16-alpine` sera tirée de nouveau.

Fichiers fournis (`files/`) — aucun : tout se crée au terminal.

Étape 1 — La même image, d'autres valeurs

```
mkdir -p ~/labo-docker/08 && cd ~/labo-docker/08
podman run -d --name api -p 18080:8080 api:1.0 ; sleep 2
curl -s localhost:18080/ ; echo
podman rm -f -t 0 api
podman run -d --name api -p 18080:8080 -p 19090:9090 -e APP_MESSAGE="config par -e" -e
APP_PROFILE=prod -e SERVER_PORT=9090 api:1.0 ; sleep 2
curl -s localhost:18080/ ; echo
curl -s localhost:19090/ ; echo
podman logs api | head -2
```

Observez d'abord `{"message":"Bonjour depuis l'API","profile":"default"}` — les défauts codés dans l'application —, puis **rien** sur 18080 et `{"message":"config par -e","profile":"prod"}` sur 19090, avec `API demarree sur le port 9090 (profil prod)` dans les logs.

Explication. Trois variables ont changé le message, le profil et le port d'écoute, sans rebuild. Et `-p 18080:8080` relaie vers un port où plus personne n'écoute : `EXPOSE 8080` dans l'image n'a rien vérifié.

Étape 2 — Tout se voit

```
podman inspect api --format '{{range .Config.Env}}{{println .}}{{end}}' | grep APP_
podman exec api env | grep APP_
podman exec api sh -c 'tr "\0" "\n" < /proc/1/environ | grep APP_'
podman rm -f -t 0 api
```

Observez trois fois les mêmes `APP_MESSAGE=config par -e` et `APP_PROFILE=prod` : dans les métadonnées du conteneur, dans l'environnement vu par `exec`, et dans celui du processus 1.

```
podman run -d --name api api:1.0 --db-password=Secr3t ; sleep 2
podman logs api | head -1
podman inspect api --format '{{json .Config.Cmd}}'
podman top api args
ps -eo args | grep -c 'db-password=Secr3t'
podman rm -f -t 0 api
```

Observez `Arguments recus : --db-password=Secr3t` dans les **logs**, `["--db-password=Secr3t"]` dans l'inspect, `java -jar /app/api.jar --db-password=Secr3t` dans `top`, et `2` processus de l'hôte qui affichent la valeur (le `java` et le `grep` lui-même).

Explication. Une variable est lisible par qui a accès au moteur ; un argument l'est par **tout le monde** sur la machine, et il finit dans les logs. Aucun des deux n'est un coffre.

Étape 3 — Priorité des sources

```
printf '# configuration de l API (jamais dans git)\nAPP_MESSAGE=config par env-file\nAPP_PROFILE=staging\n' > api.env
printf 'FROM localhost/api:1.0\nENV APP_MESSAGE="config depuis l image" APP_PROFILE=image\n' > Dockerfile.env
podman build -q -f Dockerfile.env -t api:env .
podman image inspect api:env --format '{{range .Config.Env}}{{println .}}{{end}}' | grep APP_
```

Observez `APP_MESSAGE=config depuis l image` et `APP_PROFILE=image` : des valeurs par défaut inscrites dans l'image.

```
podman run -d -p 18080:8080 --name a1 api:env ; sleep 2 ; curl -s localhost:18080/ ; echo ; podman rm -f -t 0 a1
podman run -d -p 18080:8080 --name a2 --env-file api.env api:env ; sleep 2 ; curl -s localhost:18080/ ; echo ; podman rm -f -t 0 a2
podman run -d -p 18080:8080 --name a3 -e APP_PROFILE=prod --env-file api.env api:env ; sleep 2 ; curl -s localhost:18080/ ; echo ; podman rm -f -t 0 a3
APP_MESSAGE="depuis le shell de l hote" podman run -d -p 18080:8080 --name a4 -e APP_MESSAGE api:env ; sleep 2 ; curl -s localhost:18080/ ; echo ; podman rm -f -t 0 a4
```

Observez successivement `"config depuis l image"/"image"` (l'image seule), `"config par env-file"/"staging"` (le fichier écrase l'image), `"config par env-file"/"prod"` (`-e` écrase le fichier, bien qu'écrit **avant** `--env-file`), et `"depuis le shell de l hote"/"image"` (`-e APP_MESSAGE` sans valeur a copié la variable du shell).

Explication. La précédence est par catégorie, pas par position : `-e` > `--env-file` > `ENV`. Le fichier `api.env` est le candidat naturel pour « un jeu de valeurs par environnement », à condition de rester hors de Git.

Étape 4 — La JVM lit aussi l'environnement

```
podman run -d --name a5 -e JAVA_TOOL_OPTIONS="-Xmx64m" api:1.0 ; sleep 2
podman logs a5 2>&1 | head -1
podman rm -f -t 0 a5
```

Observez `Picked up JAVA_TOOL_OPTIONS: -Xmx64m` : la JVM a pris l'option sans que l'`ENTRYPOINT` change.

Explication. C'est le levier standard pour régler la mémoire ou brancher un agent sur une image que l'on ne reconstruit pas.

Étape 5 — Un secret du moteur pour PostgreSQL

```
printf 'S3cr3t-pg' | podman secret create db_password -
podman secret ls
podman secret inspect db_password --format '{{.Spec.Driver.Name}}
{{.Spec.Driver.Options.path}}'
podman secret inspect --showsecret db_password --format '{{.SecretData}}'
```

Observez un ID, `db_password` avec le pilote `file`, le chemin `…/storage/secrets/filedriver`, et — avec `--showsecret` — `S3cr3t-pg` : rangé, pas chiffré.

```
podman run -d --name db --secret db_password -e POSTGRES_PASSWORD_FILE=/run/secrets/db_password
postgres:16-alpine ; sleep 4
podman exec db ls -l /run/secrets/
podman inspect db --format '{{range .Config.Env}}{{println .}}{{end}}' | grep POSTGRES
podman inspect db --format '{{json .Config.Env}}' | grep -c 'S3cr3t'
podman run --rm --network container:db -e PGPASSWORD=S3cr3t-pg postgres:16-alpine psql -h
localhost -U postgres -c 'select current_user'
```

Observez `-r--r--r-- 1 root root 9 … db_password`, un seul `POSTGRES_PASSWORD_FILE=/run/secrets/db_password` dans l'environnement, `0` occurrence de la valeur dans l'inspect, et `current_user = postgres` : le mot de passe est bien celui du secret.

```
podman rm -f -t 0 db
podman run -d --name db2 --secret db_password,type=env,target=POSTGRES_PASSWORD postgres:16-
alpine ; sleep 3
podman inspect db2 --format '{{json .Config.Env}}' | grep -c POSTGRES_PASSWORD
podman exec db2 env | grep POSTGRES_PASSWORD
podman rm -f -t 0 db2
```

Observez `0` dans l'inspect mais `POSTGRES_PASSWORD=S3cr3t-pg` dans l'environnement du processus : la variante `type=env` sert aux applications qui ne lisent pas de fichier, au prix d'une valeur visible par `exec`.

Explication. Le secret ne transite ni par la ligne de commande ni par les métadonnées. Avec `_FILE`, seul le processus qui ouvre `/run/secrets/db_password` le voit.

Étape 6 — Un secret au build

```
printf 'ghp_jeton-prive' > token.txt
printf 'FROM docker.io/library/alpine\nRUN --mount=type=secret,id=token,required=true wc -c
/run/secrets/token && echo "jeton utilise, pas conserve"\nRUN ls /run/secrets 2>&1 || true\n' >
Dockerfile.secret
podman build --secret id=token,src=token.txt -f Dockerfile.secret -t secret-demo . 2>&1 | grep
-v '^-->'
podman history --no-trunc secret-demo --format '{{.CreatedBy}}' | head -2
podman run --rm secret-demo ls /run/secrets
```

Observez `15 /run/secrets/token` pendant l'instruction qui monte le secret, puis `ls: /run/secrets: No such file or directory` dès l'instruction suivante ; dans l'historique, la ligne `--mount=type=secret,id=token,…` **sans la valeur** ; et dans l'image finale, pas de `/run/secrets`.

```
podman build -f Dockerfile.secret -t secret-demo2 . 2>&1 | tail -1
podman build --no-cache -f Dockerfile.secret -t secret-demo2 . 2>&1 | tail -1
```

Observez que le build **sans** `--secret` réussit (tout vient du cache), puis, avec `--no-cache`, `Error: … secret required but no secret with id "token" found`.

Explication. Le secret n'existe que pendant une instruction, et n'entre dans aucune couche. Le cache, lui, ne redemande rien : un jeton expiré se découvre au premier build sans cache.

Nettoyage

```
podman secret rm db_password
podman rmi api:env secret-demo secret-demo2
podman rmi docker.io/library/postgres:16-alpine
rm -f token.txt Dockerfile.secret
podman secret ls ; podman ps -a --format '{{.Names}}'
```

Observez une liste de secrets vide et aucun conteneur. Gardez `api.env` et `Dockerfile.env` : le labo 09 réutilise `api.env` avec Compose.

Ce que vous devez pouvoir affirmer maintenant

- Trois `-e` ont changé message, profil et port sans rebuild — et `-p 18080:8080` a relayé dans le vide.
- Une variable se voit dans `inspect`, `exec env` et `/proc/1/environ` ; un argument se voit **en plus** dans les logs, `top` et le `ps` de l'hôte.
- `-e` > `--env-file` > `ENV`, quel que soit l'ordre ; `-e CLE` seul copie la variable de l'hôte.
- `Picked up JAVA_TOOL_OPTIONS` : la JVM se configure par variable.
- `podman secret` + `_FILE` : `0` occurrence du mot de passe dans `inspect`, un fichier `444` dans `/run/secrets/` ; `--showsecret` prouve que ce n'est pas chiffré.
- `--mount=type=secret` : `15 /run/secrets/token` pendant l'instruction, `No such file or directory` après, rien dans `history`.
- Sans `--secret`, le build en cache réussit ; `--no-cache` → `secret required but no secret with id "token" found`.